

Defect Classification Model

IN INJECTION MOLDING USING MACHINE LEARNING

Project Flow

01 Introduction

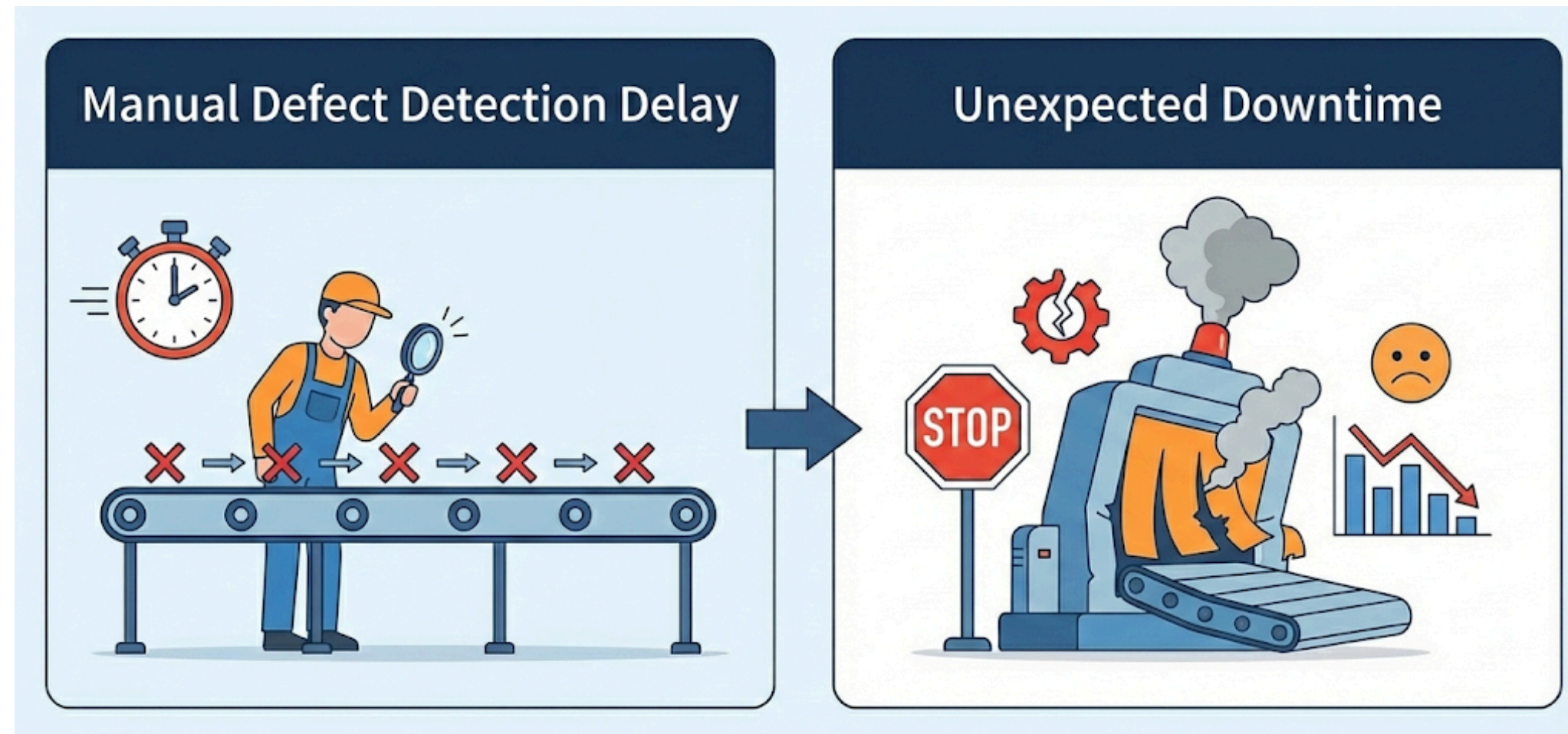
02 EDA

03 Methodology

04 Key Finding

05 Main takeaways

01 Problem Definition



Manual process monitoring causes delayed fault detection and unexpected downtime

It is essential to establish a system that can accurately assess equipment health and predict the occurrence of defective products before they happen.

Data Summary

Data Source : KAMP - injection_modeling.csv

Data Overview

Total Samples:1,323

Total Variables: 71(Columns)

Data: Process Parameters
(Pressure, Temp, Time)

Input/Output

Target(Y) : defect_binary (0:normal, 1:Defect)

Features(X):

Process Data: CV_***, SV_*** (Pressure, Temp, etc.)

Product Info: stock_mst_code

Reference: weight

But, Missing Labels

Data Summary

Motivation

- **Core Metric:** Product Weight Consistency
- **Expected Value:** Cost Reduction & Quality Optimization

Objective
“Develop a **classification model** to identify **defective** products in Injection Molding using machine learning techniques.”



Key Research Questions

- Q1 (Data):** What patterns (e.g., scale diversity, imbalance) can be derived from the process data?
- Q2 (Model):** Which ML algorithm yields the best performance for defect detection?
- Q3 (Explainability):** Which process parameters (e.g., Temp, Pressure) are the critical drivers of defects?

02 EDA

Defect Definition



Problem

Absence of ground truth labels in the raw dataset.

Solution

Applied Statistical Process Control (SPC) techniques to generate pseudo-labels for supervised learning.

Normal

Data points within the **Mean** $\pm 2\sigma$ range.

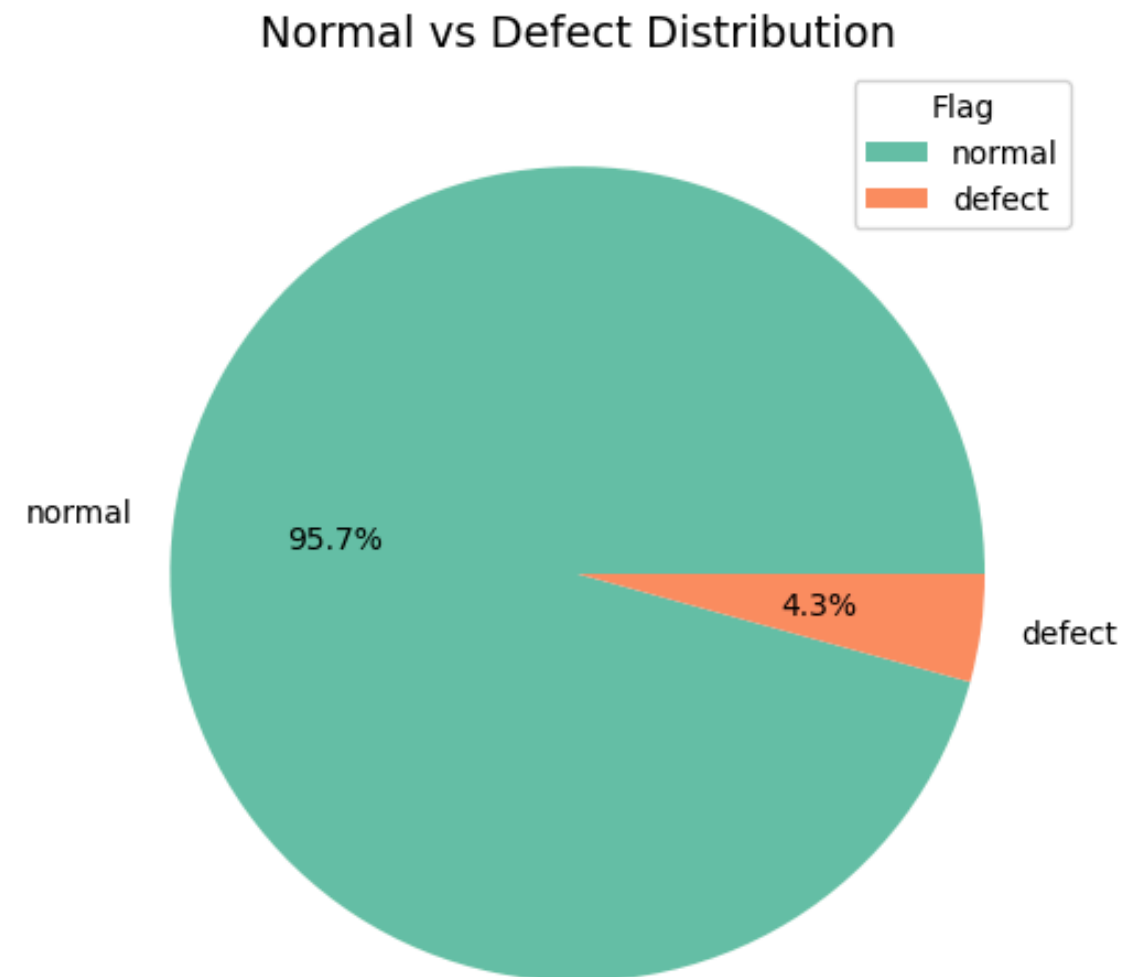
Defect (Outlier): Data points falling outside this threshold.

Filtering

Excluded product groups with insufficient samples ($n < 10$) to ensure statistical validity.

EDA

Class Imbalance



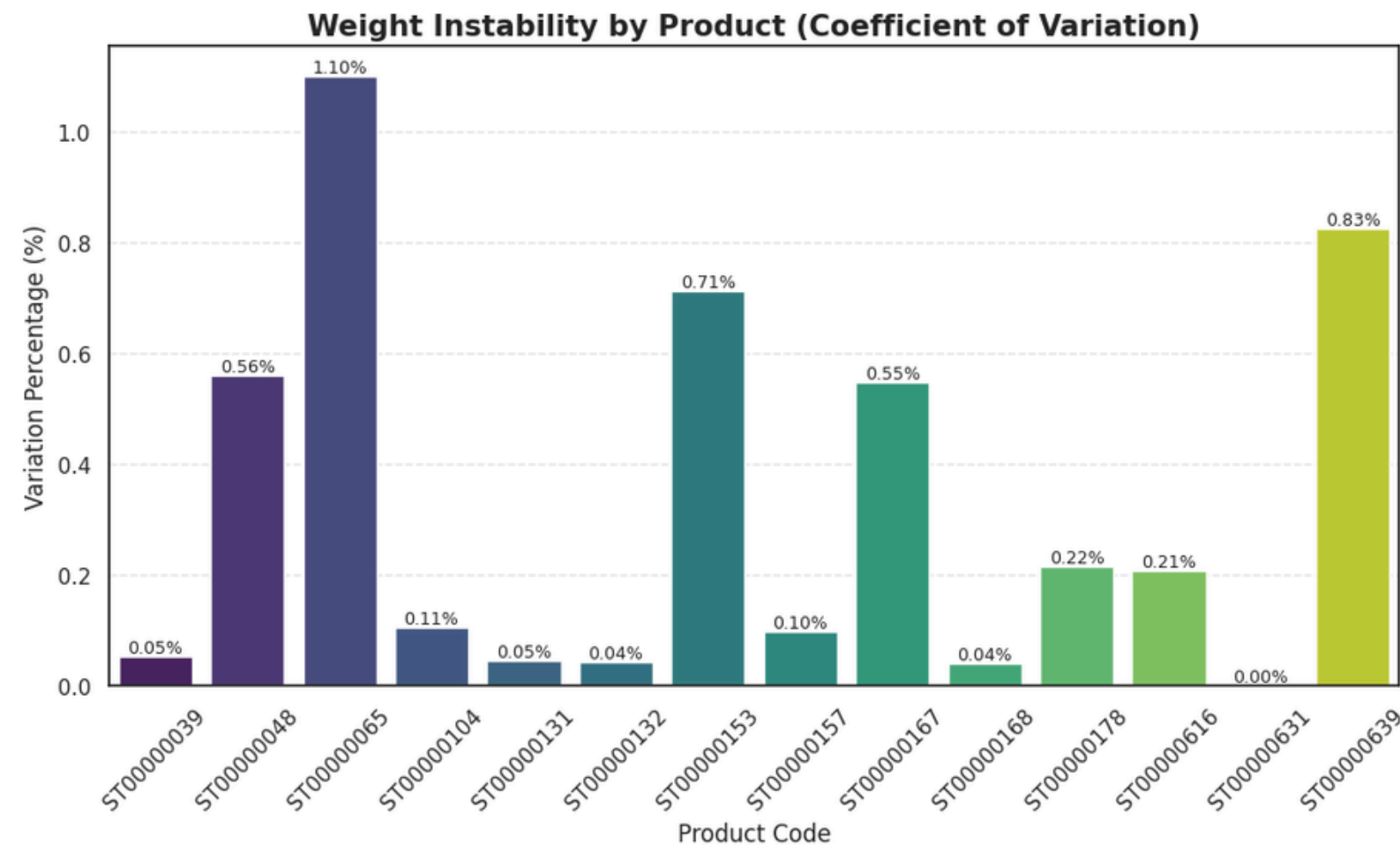
Finding

Normal: 95.7% **Defect: 4.3%**

The dataset is highly **imbalanced**.
Simple Accuracy is not a valid metric
F1-Score should be used, and handling
techniques like
Oversampling are required.

EDA

Product Diversity



1. Why CV (Coefficient of Variation)?

Due to significant scale differences in product weights, standard deviation alone is insufficient for comparison.

Metric $CV(\%) = \left(\frac{\sigma}{\mu} \right) \times 100$

2. Key Findings

Highest Instability: Product ST00000065 shows the highest variation rate (1.10%), indicating a need for stricter process control.

Stability: In contrast, products like ST00000631 show near-zero variation (0.00%), reflecting a highly stable process.

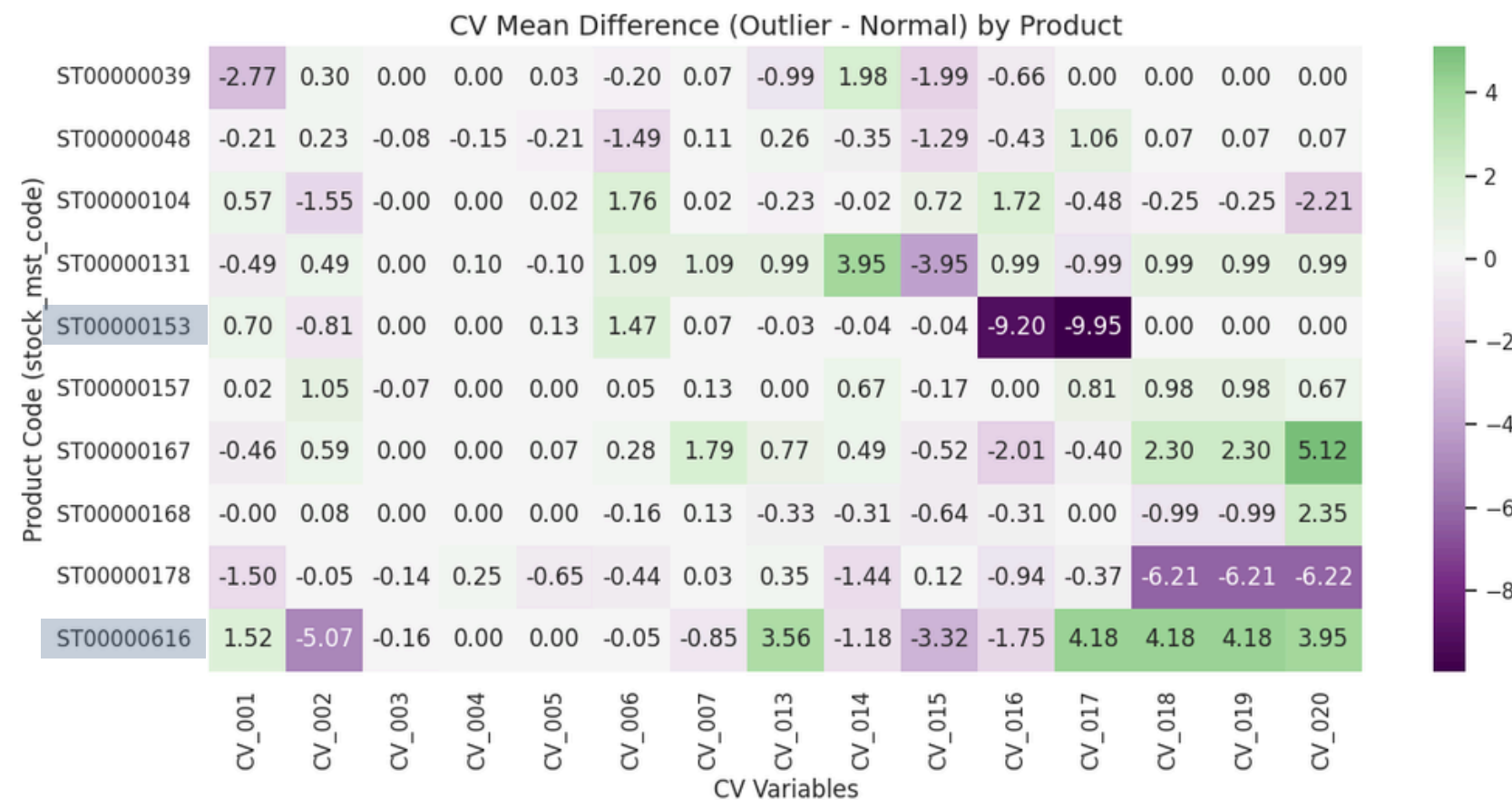
3. Insight

Significant differences in stability across products confirm that product-specific scaling is essential for model performance.

EDA

Correlation Heatmap(Normal vs Defect)

Visualized the mean difference between Defective and Normal products.



- **Under-heating ST0000153**

Significant negative deviation in CV_016 & CV_017 (Temp Zones 1-3, 1-4)]

Root Cause: Defects occur due to a sudden temperature drop in the middle barrel zones.

- **Complex Failure ST0000616**

Negative deviation in CV_002 (Mold Open Position) combined with positive deviation in CV_017~CV_020 (Temp Zones 1-4 to 1-7).

Root Cause: A mix of insufficient mold opening and overheating leads to defects.

Strategic Implication

Since failure triggers (e.g., Thermal vs. Mechanical) vary by product, the model must learn non-linear, product-specific relationships.

03 Methodology

Methodology Overview

- Feature Engineering
- Feature Selection
- Handling Imbalance
- Preprocessing Pipelines
- Candidate Models
- Model Selection & Validation

Feature Engineering & Target Definition

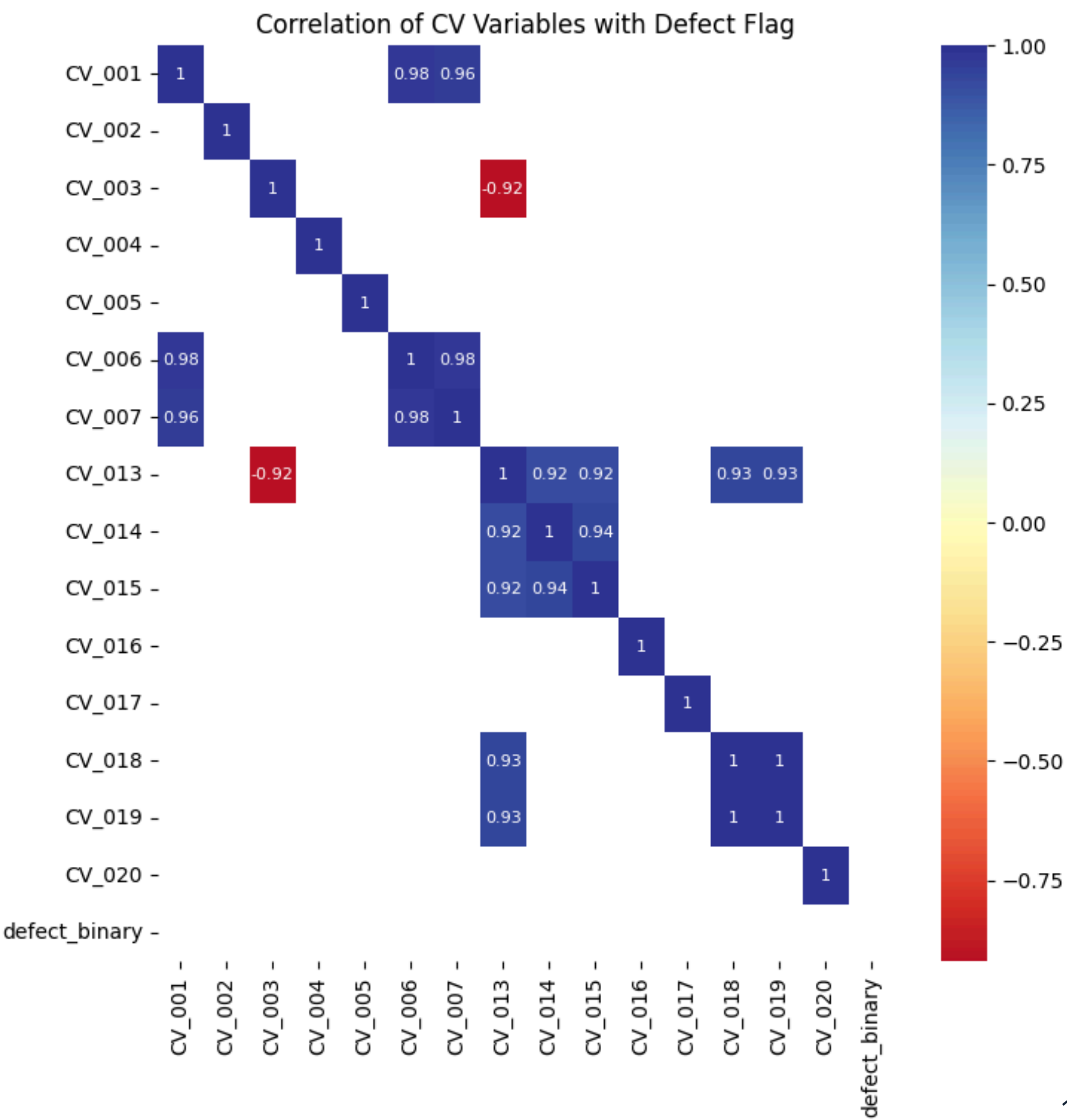
EDA finding	Engineering decision
Sensors show no NA but some MNAR patterns (inactive sensors)	Keep variables but monitor impact during modeling
Weight varies by product group	Define defects using $\pm 2\sigma$ per product group
defect_flag is noisy/continuous	Convert to clean binary target: defect_binary (0/1)
Metadata categories exist (stock_mst_code, MOLD_CAVITY)	Encode categorical features for model input
Class imbalance: only 4.3% defects	Apply RandomOverSampler during training

Feature Selection (CV & SV)

- Correlation analysis used to identify redundant clusters
- Representative features selected from each correlation cluster
- Final dataset includes numeric + categorical features

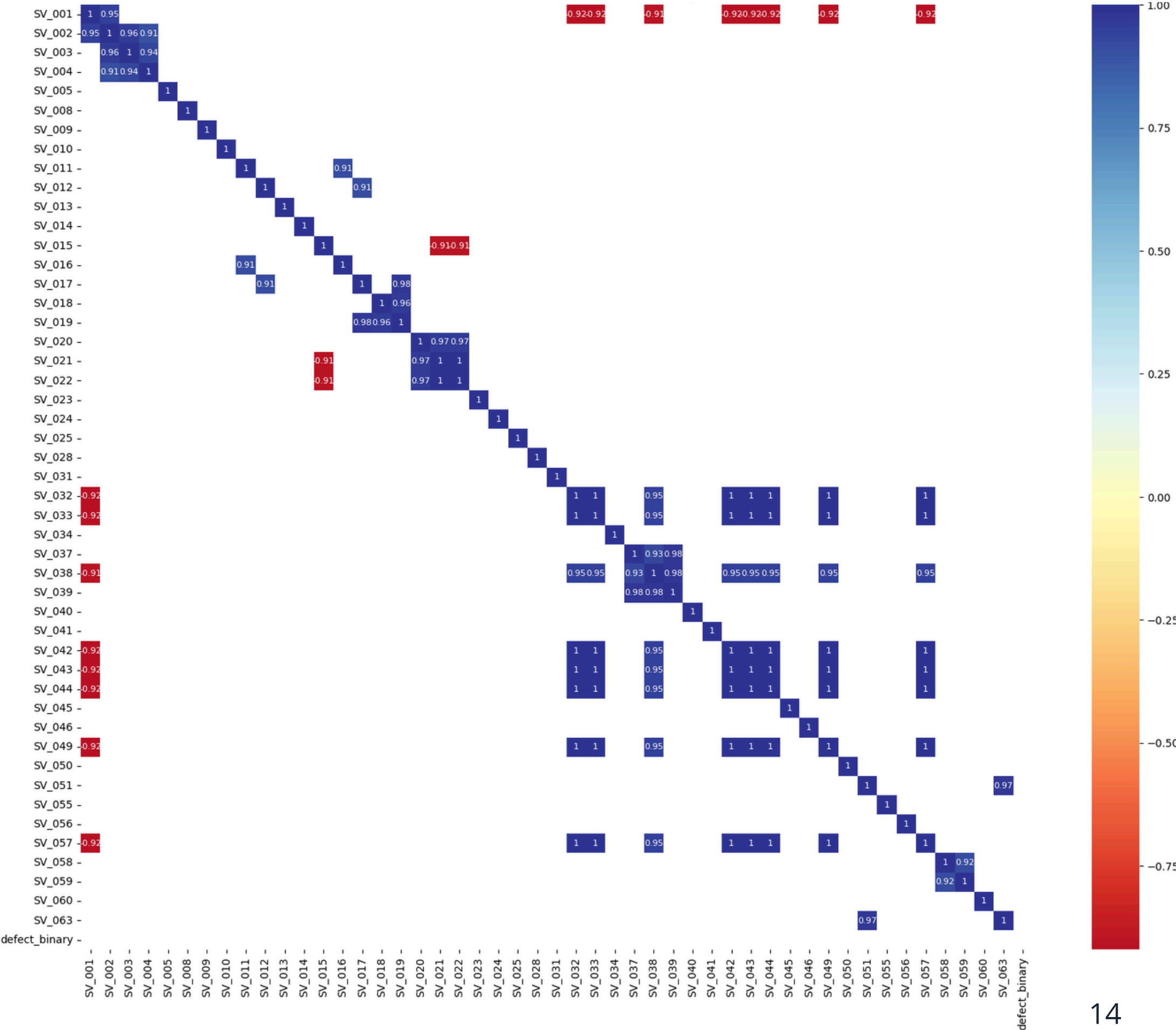
Feature Selection (CV)

Selected CV features (8)	
CV_003	CV_015
CV_004	CV_016
CV_005	CV_018
CV_006	CV_020



Feature Selection (SV)

Selected SV features (12)	
SV_003	SV_042
SV_013	SV_049
SV_018	SV_055
SV_023	SV_057
SV_031	SV_059
SV_037	SV_063



Building the Dataset

- From 71 fields → 22 fields
- 80% Train, 20%Test, stratified
- Due to the imbalance, there is a need to balance the Dataset

Dataset	y_train dist	y_train pct	y_train dist	y_train pct
defect = 0	1002	50	251	95.8
defect = 1	1002	50	11	4.2

Data Modeling

Preprocessing Steps

- Numerical:
 - StandardScaler
 - MinMaxScaler
 - PowerTransformer
 - Polynomial Features
 - Spline Transformer
- Categorical:
 - One-Hot Encoding

Modeling

- Logistic Regression
- KNN
- Decision Tree
- Random Forest
- XGBoost

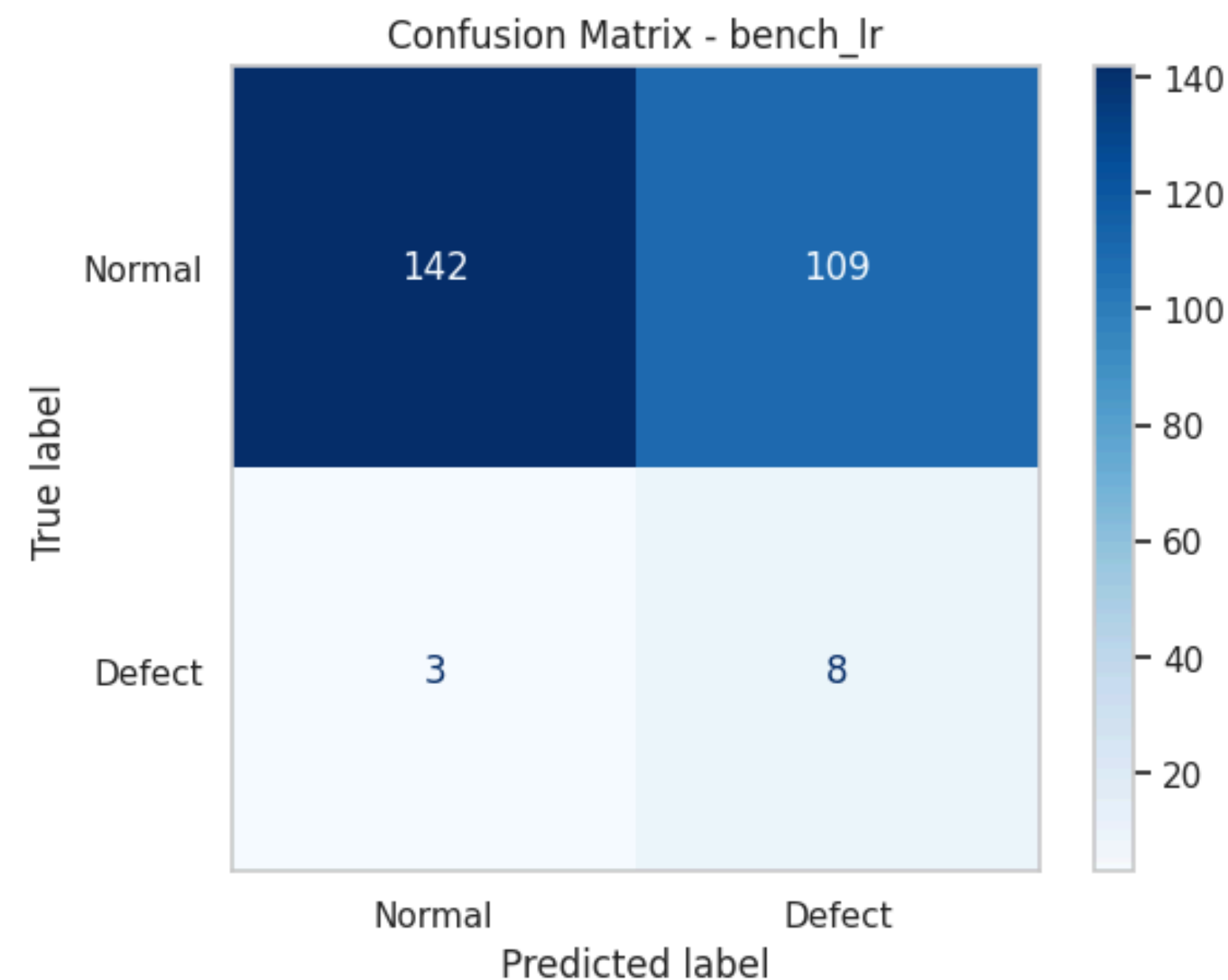
Benchmarking a Baseline Model

Trained using all CV/SV variables

Pipeline:

- OneHotEncoder
- StandardScaler
- Logistic Regression

	Accuracy	F1	AUC	recall
Bench_Ir	0.57	0.125	0.76	0.73



Pipeline Definition

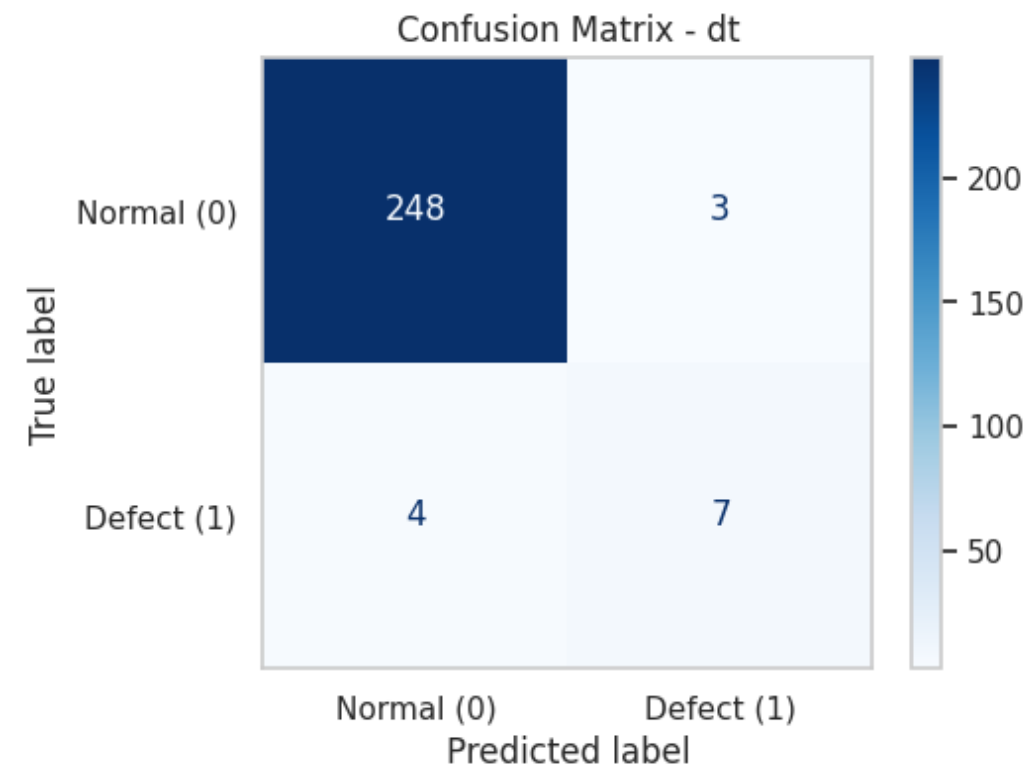
Model Type	Algorithm	Key Preprocessing (Strategy)
Linear Model	Logistic Regression	Standard / MinMax Scaling
	Logistic Regression	Power Transformer (Skewness)
	Logistic Regression	Spline / Poly Features (Non-linear)
Simple Model	KNN	Spline Transformer
	Decision Tree	Polynomial Features
Tree Ensemble	Random Forest	One-Hot Encoding (No Scaling)
	XGBoost	One-Hot Encoding (No Scaling)

Comparison of Model Results

Model	accuracy	precision	recall	f1
bench_lr	0.572519	0.068376	0.727273	0.125
lr-standard	0.641221	0.080808	0.727273	0.145455
lr-minmax	0.587786	0.070796	0.727273	0.129032
lr-power	0.637405	0.071429	0.636364	0.12844
lr-poly	0.931298	0.347826	0.727273	0.470588
lr-spline	0.839695	0.183673	0.818182	0.3
knn-spl	0.958015	0.5	0.818182	0.62069
dt-poly	0.973282	0.7	0.636364	0.666667
dt	0.618321	0.07619	0.727273	0.137931
rf	0.618321	0.07619	0.727273	0.137931
xgb	0.958015	0	0	0
xgboost_check	0.572519	0.068376	0.727273	0.125

Model Training & Validation

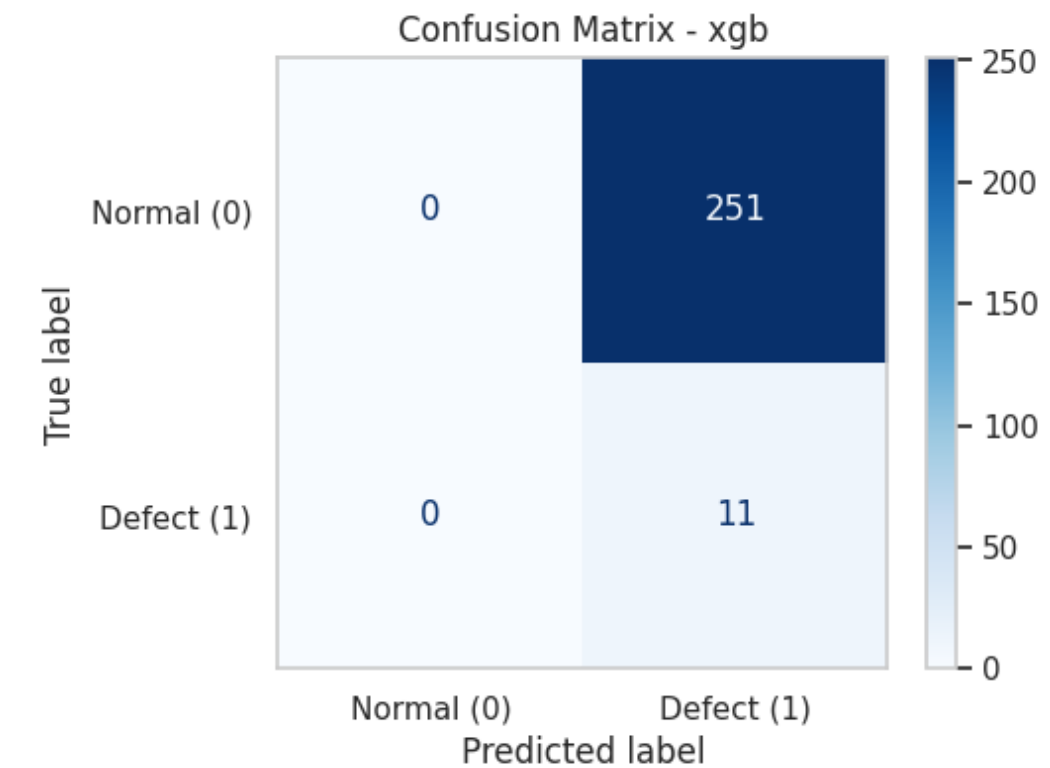
Best : Decision Tree + Poly



F1 Score 0.66

Significant increase in True Positives

Worst : XGBoost



F1 Score 0.00

Predicted all samples as 'Normal'

Model Stability in Table

Group 1 Pipeline

Preprocessing Strategy	Mean F1-Score	Std Dev (σ)	Performance
Standard Scaling	0.121	0.01	Low
MinMax Scaling	0.116	0.008	Low
Power Transformer	0.117	0.015	Low
Spline Transformer	0.161	0.013	Moderate
Polynomial Features	0.326	0.031	Best

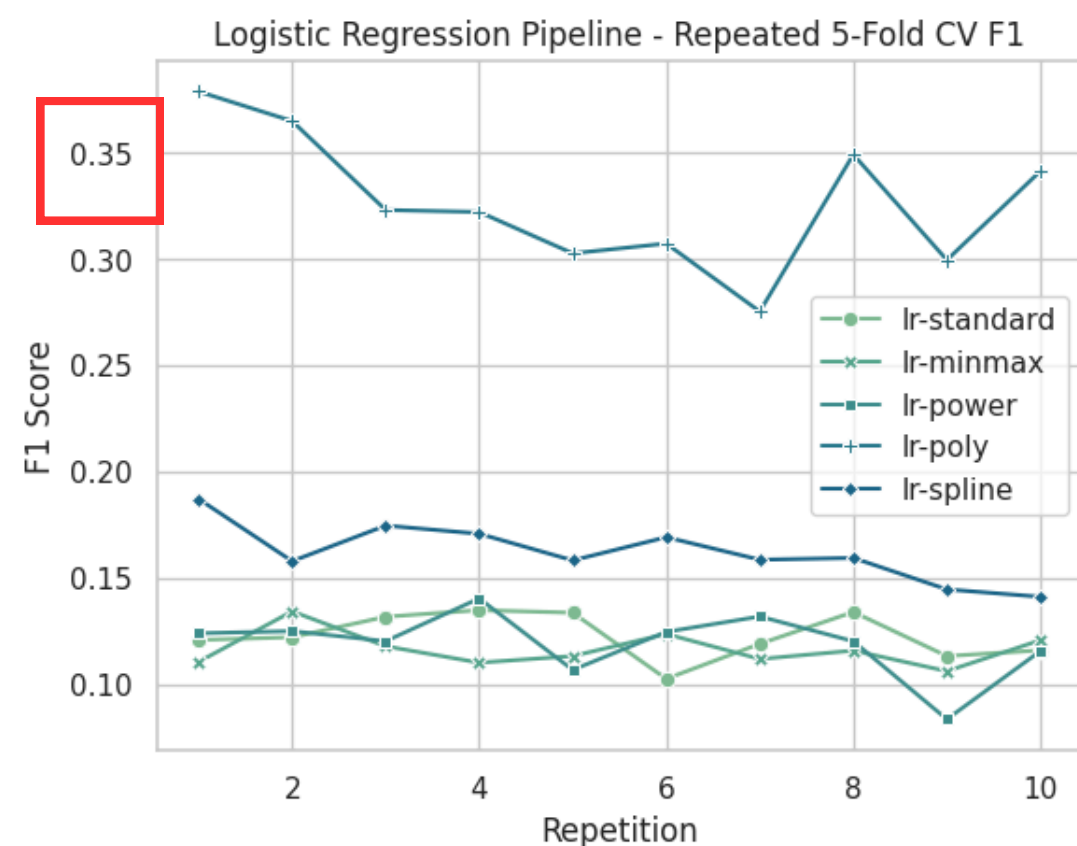
Group 2 Pipelines

Model Pipeline	Mean F1-Score	Std Dev (σ)	Performance
KNN (Spline Trans.)	0.246	0.066	Moderate
Decision Tree (Poly)	0.458	0.023	High

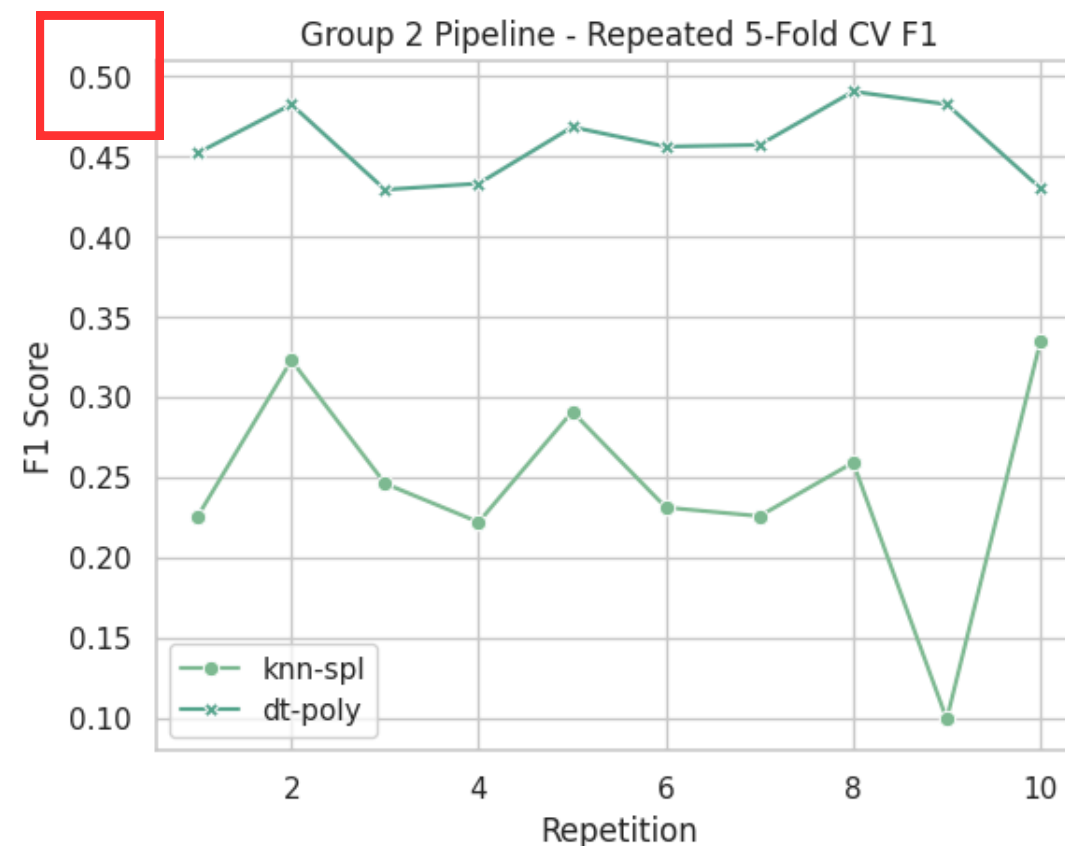
Group 3 Pipelines

Model Pipeline (One-Hot Only)	Mean F1-Score	Std Dev (σ)	Performance
Decision Tree	0.111	0.007	Low
Random Forest	0.111	0.006	Low
XGBoost	0	0	Failed

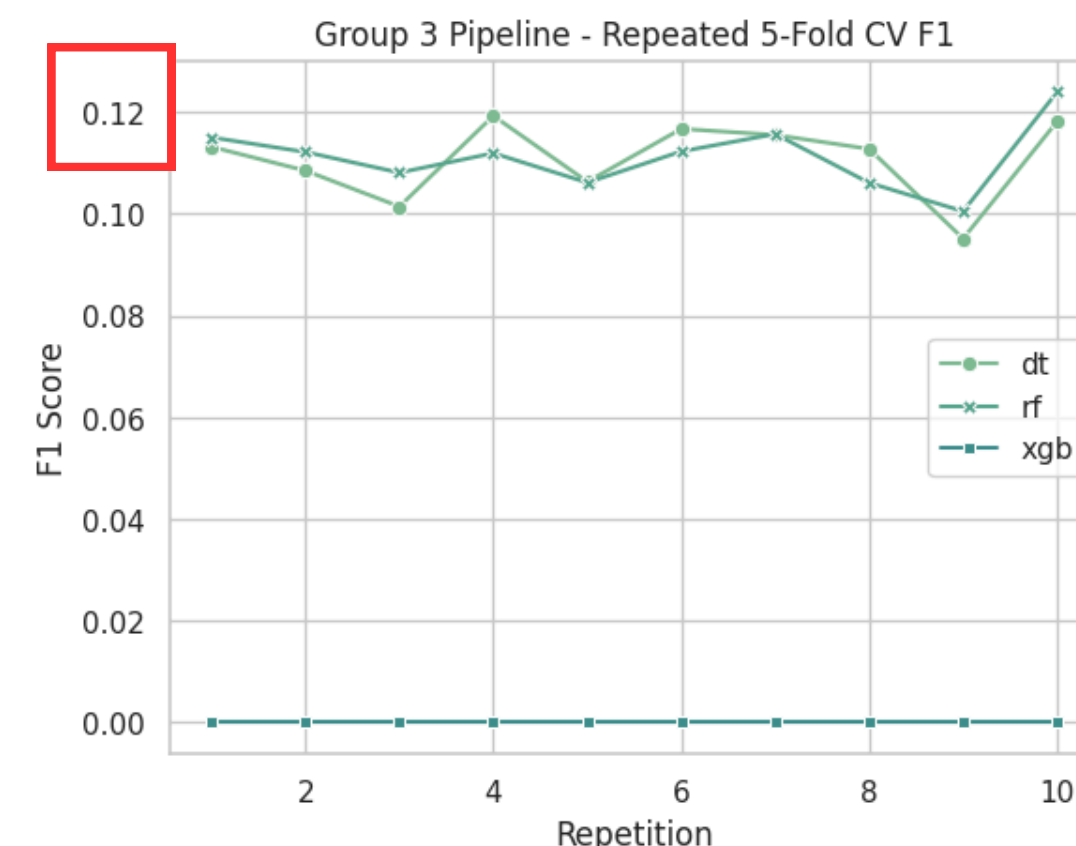
Model Stability in Graph



Linear models show low and unstable F1 (≈ 0.10 – 0.35), indicating they cannot capture nonlinear process patterns

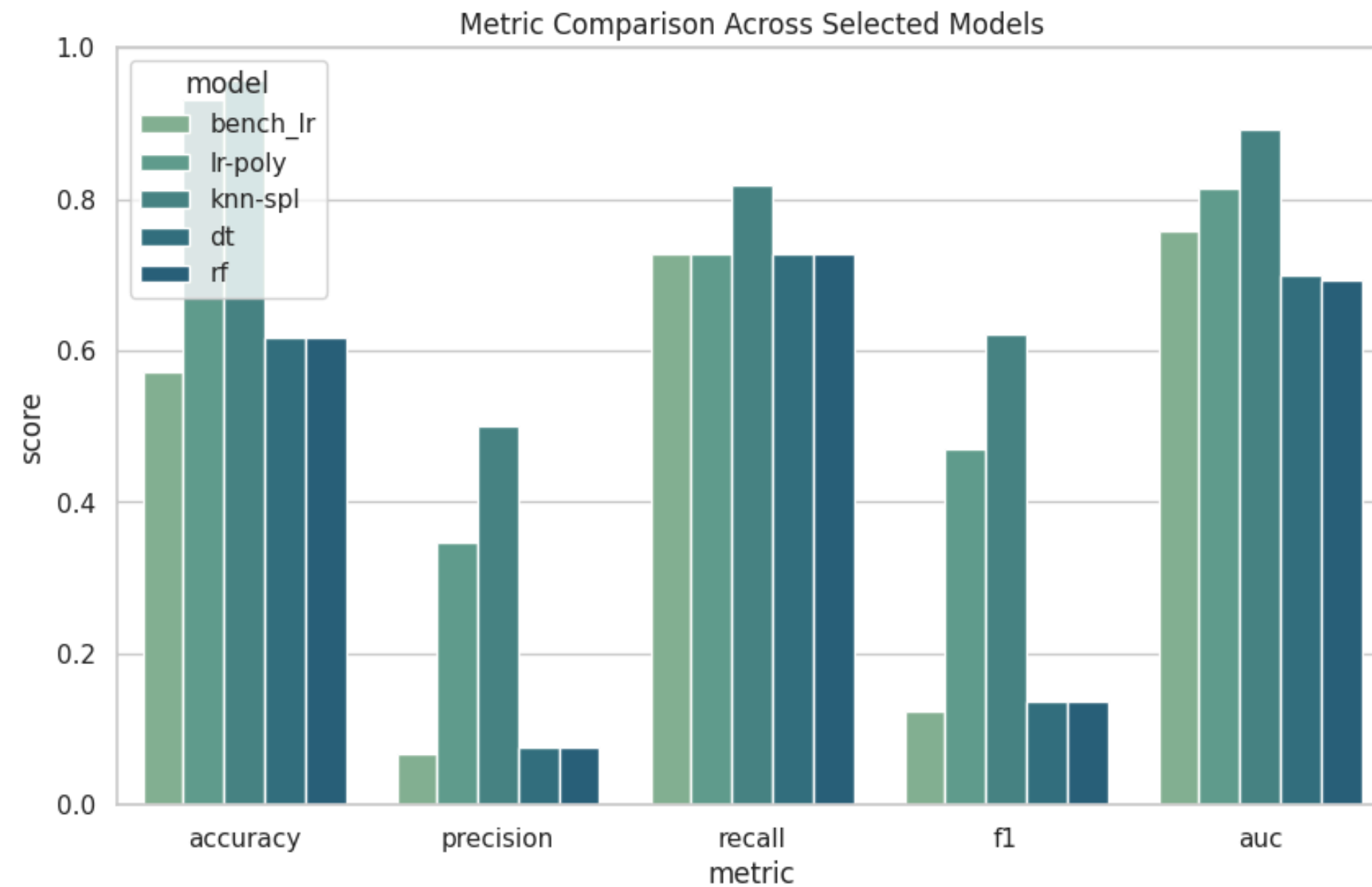


KNN-Spline consistently achieves the highest F1 (≈ 0.45 – 0.50) and shows the most stable performance across repetitions

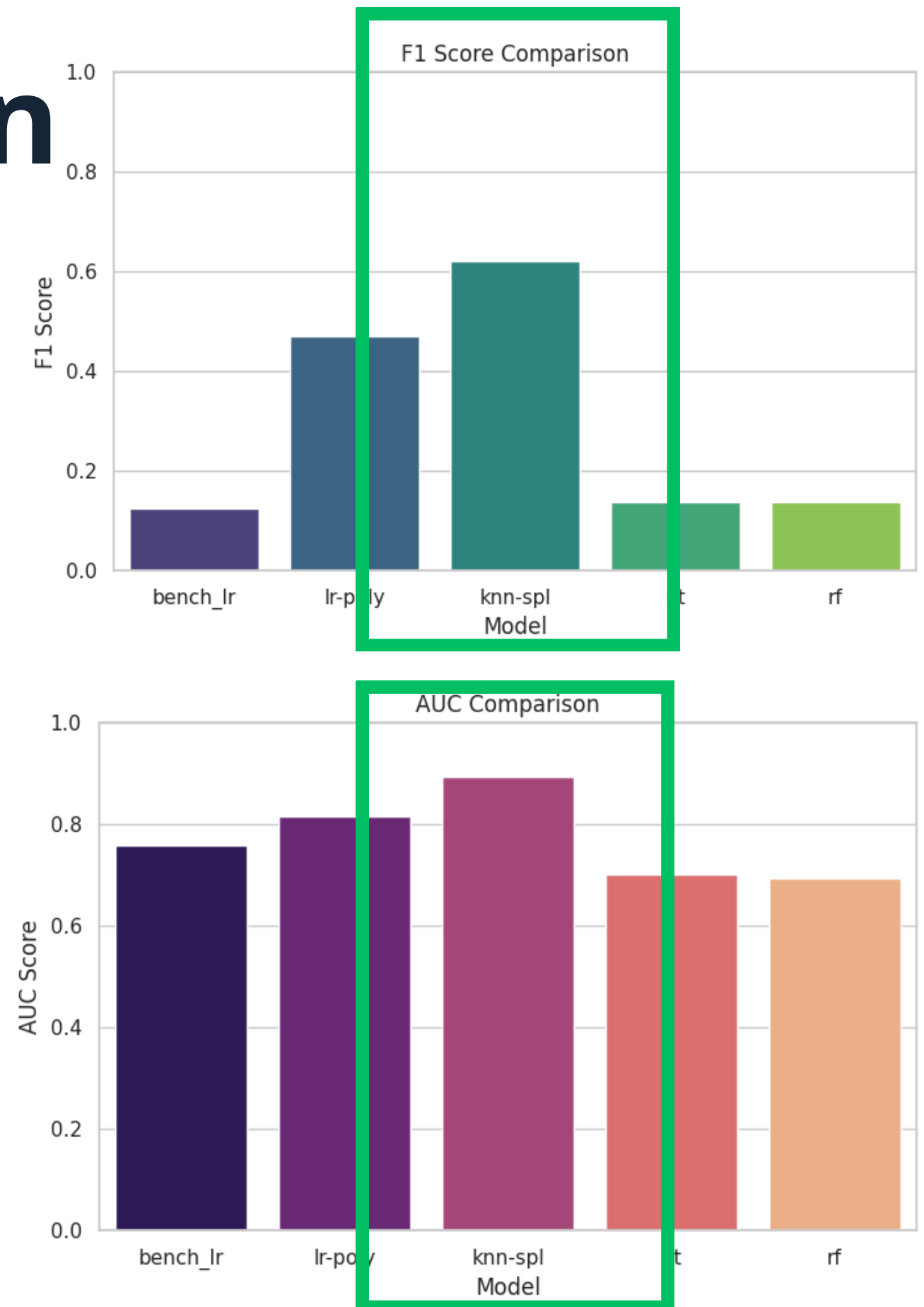


Models without numeric preprocessing collapse (F1 ≈ 0.10 – 0.12), predicting defects poorly due to imbalance and unscaled features

Final Model Comparison

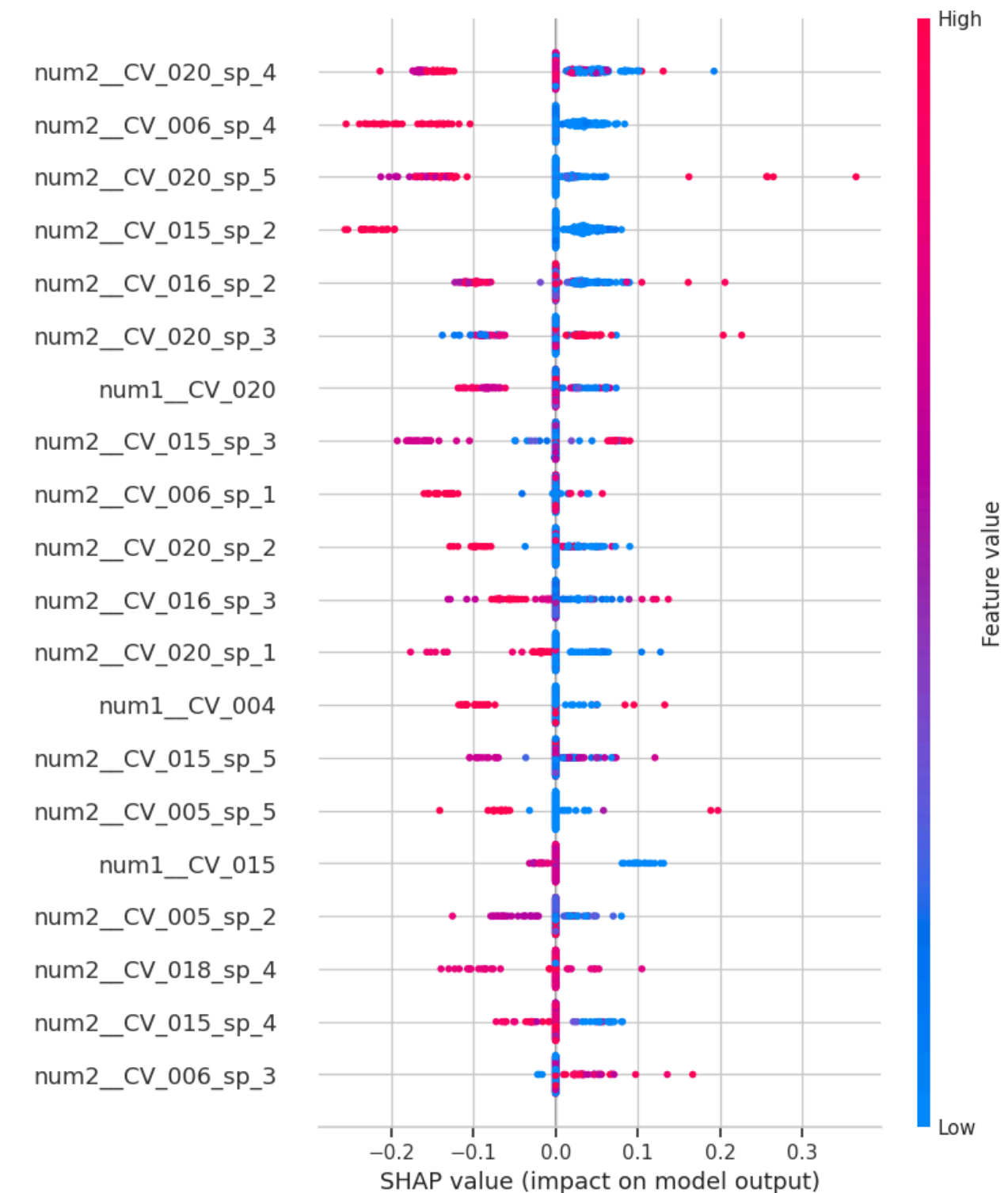


KNN-spline as the best model for this dataset as its overall performance across metric is remarkable and within standards.



Bonus - Interpretability

- The model identifies the following cycle variables (CV) as the strongest drivers of defect predictions:
 - CV_020 – multiple nonlinear segments highly influential
 - CV_006 – strong localized impact
 - CV_015 – several spline components ranked highly
 - CV_016 – moderate but consistent influence
- These CV features contain the most discriminative patterns between normal and defective cycles.



Key Insights

- Linear models do not capture the nonlinear behavior of the process
- KNN with Spline transformation achieves the best performance across all models, making it the final selected model
- Decision Tree with Polynomial Features performs strongly but still behind KNN-Spline
- Tree-based models fail when only One-Hot Encoding is applied, due to severe class imbalance and high-dimensional categorical feature expansion

Limitation

- Defect label is proxy-based ($\pm 2\sigma$ weight rule) and may not reflect true physical defects
- Low defect rate (4.3%)
- Oversampling duplicates minority data
- Did not include/consider date
- Tree based models seem to explode as they are unstable